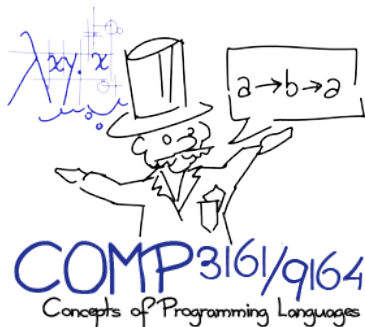




Algebraic Data Types

Craig McLaughlin
UNSW
Term 3 2023

Composite Data Types

Most of the types we have seen so far are **basic** types, in the sense that they represent built-in machine data representations.

Real programming languages have ways to **compose** types to produce new types:

Composite Data Types

Most of the types we have seen so far are **basic** types, in the sense that they represent built-in machine data representations.

Real programming languages have ways to **compose** types to produce new types:

Tuples

Structs

Records

Composite Data Types

Most of the types we have seen so far are **basic** types, in the sense that they represent built-in machine data representations. Real programming languages have ways to **compose** types to produce new types:

Classes

Tuples

Structs

Records

Composite Data Types

Most of the types we have seen so far are **basic** types, in the sense that they represent built-in machine data representations.

Real programming languages have ways to **compose** types to produce new types:

Classes

Tuples

Structs

Unions

Records

Combining values conjunctively

We want to store two things in one value.

Haskell Tuples

```
type Point = (Float, Float)
```

```
midpoint (x1,y1) (x2,y2)  
  = ((x1+x2)/2, (y1+y2)/2)
```

Combining values conjunctively

We want to store two things in one value.

Haskell Tuples

```
type Point = (Float, Float)
```

```
midpoint (x1,y1) (x2,y2)  
  = ((x1+x2)/2, (y1+y2)/2)
```

Haskell Datatypes

```
data Point =
```

```
  Pnt { x :: Float  
        , y :: Float  
        }
```

```
midpoint (Pnt x1 y1) (Pnt x2 y2)  
  = Pnt ((x1+x2)/2) ((y1+y2)/2)
```

```
midpoint' p1 p2 =  
  = Pnt ((x p1 + x p2) / 2)  
        ((y p1 + y p2) / 2)
```

Combining values conjunctively

We want to store two things in one value.

C Structs

```
typedef struct point {
```

```
    float x;
```

```
    float y;
```

```
} point;
```

```
point midpoint (point p1, point p2) {
```

```
    point mid;
```

```
    mid.x = (p1.x + p2.x) / 2.0;
```

```
    mid.y = (p2.y + p2.y) / 2.0;
```

```
    return mid;
```

```
}
```

types

Has

type Point

midpoint (

= ((x1+x2

) (Pnt x2 y2)

((y1+y2)/2)

) / 2)

) / 2)

Combining values conjunctively

We want to store two things in one value.

C Structs

types

Java

```
type Point {
  float x;
  float y;
}

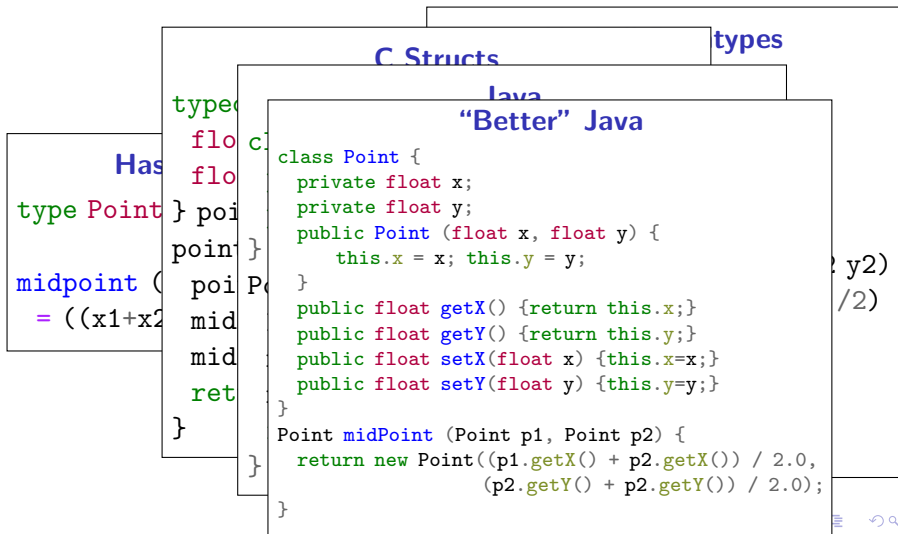
Point midpoint (Point p1, Point p2) {
  Point mid = new Point();
  mid.x = (p1.x + p2.x) / 2.0;
  mid.y = (p1.y + p2.y) / 2.0;
  return mid;
}
```

```
class Point {
  public float x;
  public float y;
}

Point midpoint (Point p1, Point p2) {
  Point mid = new Point();
  mid.x = (p1.x + p2.x) / 2.0;
  mid.y = (p1.y + p2.y) / 2.0;
  return mid;
}
```

Combining values conjunctively

We want to store two things in one value.



Product Types

In MinHS, we will have a very minimal way to accomplish this, called a *product type*:

$$\tau_1 \times \tau_2$$

We won't have type declarations, named fields or anything like that. More than two values can be combined by nesting products, for example a three dimensional vector:

$$\text{Int} \times (\text{Int} \times \text{Int})$$

Constructors and Eliminators

We can **construct** a product type similar to Haskell tuples:

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2}$$

The only way to extract each component of the product is to use the `fst` and `snd` eliminators:

$$\frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst } e : \tau_1} \quad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd } e : \tau_2}$$

Examples

Example (Midpoint)

```
recfun midpoint ::  
  ((Int × Int) → (Int × Int) → (Int × Int)) p1 =  
  recfun midpoint' ::  
    ((Int × Int) → (Int × Int)) p2 =  
    ((fst p1 + fst p2) ÷ 2, (snd p1 + snd p2) ÷ 2)
```

Example (Uncurried Division)

```
recfun div :: ((Int × Int) → Int) args =  
  if (fst args < snd args)  
    then 0  
    else 1 + div (fst args - snd args, snd args)
```

Dynamic Semantics

$$\frac{e_1 \mapsto_M e'_1}{(e_1, e_2) \mapsto_M (e'_1, e_2)}$$

$$\frac{e_2 \mapsto_M e'_2}{(v_1, e_2) \mapsto_M (v_1, e'_2)}$$

$$\frac{e \mapsto e'}{\text{fst } e \mapsto_M \text{fst } e'}$$

$$\frac{e \mapsto e'}{\text{snd } e \mapsto_M \text{snd } e'}$$

$$\frac{}{\text{fst } (v_1, v_2) \mapsto_M v_1}$$

$$\frac{}{\text{snd } (v_1, v_2) \mapsto_M v_2}$$

Unit Types

Currently, we have no way to express a type with just **one** value. This may seem useless at first, but it becomes useful in combination with other types.

We'll introduce a type, **1**, pronounced *unit*, that has exactly one inhabitant, written **()**:

$$\overline{\Gamma \vdash () : 1}$$

Disjunctive Composition

We can't, with the types we have, express a type with exactly **three** values.

Example (Trivalued type)

```
data TrafficLight = Red | Amber | Green
```

Disjunctive Composition

We can't, with the types we have, express a type with exactly **three** values.

Example (Trivalued type)

```
data TrafficLight = Red | Amber | Green
```

In general we want to express data that can be **one** of multiple **alternatives**, that contain different bits of data.

Example (More elaborate alternatives)

```
type Length = Int
type Angle = Int
data Shape = Rect Length Length
           | Circle Length | Point
           | Triangle Angle Length Length
```

This is awkward in many languages. In Java we'd have to use inheritance. In C we'd have to use unions.

Sum Types

We will use *sum types* to express the possibility that data may be one of two forms.

$$\tau_1 + \tau_2$$

This is similar to the Haskell Either type.
Our TrafficLight type can be expressed (grotesquely) as a sum of units:

$$\text{TrafficLight} \simeq 1 + (1 + 1)$$

Constructors and Elimimators for Sums

To make a value of type $\tau_1 + \tau_2$, we invoke one of two **constructors**:

$$\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{InL } e : \tau_1 + \tau_2} \quad \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{InR } e : \tau_1 + \tau_2}$$

We can branch based on which alternative is used using **pattern matching**:

$$\frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad x : \tau_1, \Gamma \vdash e_1 : \tau \quad y : \tau_2, \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\text{case } e \text{ of InL } x \rightarrow e_1; \text{InR } y \rightarrow e_2) : \tau}$$

Examples

Example (Traffic Lights)

Our traffic light type has three values as required:

$$\text{TrafficLight} \simeq 1 + (1 + 1)$$

$$\text{Red} \simeq \text{InL } ()$$

$$\text{Amber} \simeq \text{InR } (\text{InL } ())$$

$$\text{Green} \simeq \text{InR } (\text{InR } ())$$

Examples

We can convert most (non-recursive) Haskell types to equivalent MinHS types now.

- 1 Replace all constructors with 1
- 2 Add a $\times$ between all constructor arguments.
- 3 Change the $|$ character that separates constructors to a $+$.

Example

```
data Shape = Rect Length Length
           | Circle Length | Point
           | Triangle Angle Length Length
```

$\simeq$

$$\begin{aligned} & 1 \times (\text{Int} \times \text{Int}) \\ + & 1 \times \text{Int} + 1 \\ + & 1 \times (\text{Int} \times (\text{Int} \times \text{Int})) \end{aligned}$$

Dynamic Semantics

$$\frac{e \mapsto_M e'}{\text{InL } e \mapsto_M \text{InL } e'} \quad \frac{e \mapsto_M e'}{\text{InR } e \mapsto_M \text{InR } e'}$$

$$\frac{e \mapsto_M e'}{(\text{case } e \text{ of InL } x. e_1; \text{InR } y. e_2) \mapsto_M (\text{case } e' \text{ of InL } x. e_1; \text{InR } y. e_2)}$$

$$\frac{}{(\text{case } (\text{InL } v) \text{ of InL } x. e_1; \text{InR } y. e_2) \mapsto_M e_1[x := v]}$$

$$\frac{}{(\text{case } (\text{InR } v) \text{ of InL } x. e_1; \text{InR } y. e_2) \mapsto_M e_2[y := v]}$$

The Empty Type

We add another type, called 0 , that has **no** inhabitants. Because it is empty, there is no way to construct it.

We do have a way to eliminate it, however:

$$\frac{\Gamma \vdash e : 0}{\Gamma \vdash \text{absurd } e : ?}$$

The Empty Type

We add another type, called 0 , that has **no** inhabitants. Because it is empty, there is no way to construct it.

We do have a way to eliminate it, however:

$$\frac{\Gamma \vdash e : 0}{\Gamma \vdash \text{absurd } e : \tau}$$

If a variable of the **empty** type is in scope, we must be looking at an expression that will **never** be evaluated. Therefore, we can assign any type we like to this expression, because it will never be executed.

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$
- Commutativity: $\tau_1 + \tau_2 \simeq \tau_2 + \tau_1$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$
- Commutativity: $\tau_1 + \tau_2 \simeq \tau_2 + \tau_1$

Laws for $(\tau, \times, 1)$

- Associativity: $(\tau_1 \times \tau_2) \times \tau_3 \simeq \tau_1 \times (\tau_2 \times \tau_3)$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$
- Commutativity: $\tau_1 + \tau_2 \simeq \tau_2 + \tau_1$

Laws for $(\tau, \times, 1)$

- Associativity: $(\tau_1 \times \tau_2) \times \tau_3 \simeq \tau_1 \times (\tau_2 \times \tau_3)$
- Identity: $1 \times \tau \simeq \tau$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$
- Commutativity: $\tau_1 + \tau_2 \simeq \tau_2 + \tau_1$

Laws for $(\tau, \times, 1)$

- Associativity: $(\tau_1 \times \tau_2) \times \tau_3 \simeq \tau_1 \times (\tau_2 \times \tau_3)$
- Identity: $1 \times \tau \simeq \tau$
- Commutativity: $\tau_1 \times \tau_2 \simeq \tau_2 \times \tau_1$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$
- Commutativity: $\tau_1 + \tau_2 \simeq \tau_2 + \tau_1$

Laws for $(\tau, \times, 1)$

- Associativity: $(\tau_1 \times \tau_2) \times \tau_3 \simeq \tau_1 \times (\tau_2 \times \tau_3)$
- Identity: $1 \times \tau \simeq \tau$
- Commutativity: $\tau_1 \times \tau_2 \simeq \tau_2 \times \tau_1$

Combining $\times$ and $+$:

- Distributivity: $\tau_1 \times (\tau_2 + \tau_3) \simeq (\tau_1 \times \tau_2) + (\tau_1 \times \tau_3)$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$
- Commutativity: $\tau_1 + \tau_2 \simeq \tau_2 + \tau_1$

Laws for $(\tau, \times, 1)$

- Associativity: $(\tau_1 \times \tau_2) \times \tau_3 \simeq \tau_1 \times (\tau_2 \times \tau_3)$
- Identity: $1 \times \tau \simeq \tau$
- Commutativity: $\tau_1 \times \tau_2 \simeq \tau_2 \times \tau_1$

Combining $\times$ and $+$:

- Distributivity: $\tau_1 \times (\tau_2 + \tau_3) \simeq (\tau_1 \times \tau_2) + (\tau_1 \times \tau_3)$
- Absorption: $0 \times \tau \simeq 0$

Semiring Structure

The types we have defined form an algebraic structure called a *commutative semiring*.

Laws for $(\tau, +, 0)$:

- Associativity: $(\tau_1 + \tau_2) + \tau_3 \simeq \tau_1 + (\tau_2 + \tau_3)$
- Identity: $0 + \tau \simeq \tau$
- Commutativity: $\tau_1 + \tau_2 \simeq \tau_2 + \tau_1$

Laws for $(\tau, \times, 1)$

- Associativity: $(\tau_1 \times \tau_2) \times \tau_3 \simeq \tau_1 \times (\tau_2 \times \tau_3)$
- Identity: $1 \times \tau \simeq \tau$
- Commutativity: $\tau_1 \times \tau_2 \simeq \tau_2 \times \tau_1$

Combining $\times$ and $+$:

- Distributivity: $\tau_1 \times (\tau_2 + \tau_3) \simeq (\tau_1 \times \tau_2) + (\tau_1 \times \tau_3)$
- Absorption: $\mathbf{0} \times \tau \simeq 0$

What does $\simeq$ mean here?

Isomorphism

Two types τ_1 and τ_2 are *isomorphic*, written $\tau_1 \simeq \tau_2$, if there exists a *bijection* between them. This means that for each value in τ_1 we can find a unique value in τ_2 and vice versa.

We can use isomorphisms to simplify our Shape type:

$$\begin{aligned} & 1 \times (\text{Int} \times \text{Int}) \\ + & 1 \times \text{Int} + 1 \\ + & 1 \times (\text{Int} \times (\text{Int} \times \text{Int})) \end{aligned}$$

$$\simeq$$

$$\begin{aligned} & \text{Int} \times \text{Int} \\ + & \text{Int} + 1 \\ + & \text{Int} \times (\text{Int} \times \text{Int}) \end{aligned}$$

Examining our Types

Lets look at the rules for typed lambda calculus extended with sums and products:

$$\begin{array}{c}
 \frac{}{\Gamma \vdash e : 0} \\
 \hline
 \Gamma \vdash \text{absurd } e : \tau
 \end{array}
 \qquad
 \frac{}{\Gamma \vdash () : 1}$$

$$\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{InL } e : \tau_1 + \tau_2}
 \qquad
 \frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{InR } e : \tau_1 + \tau_2}$$

$$\frac{\Gamma \vdash e : \tau_1 + \tau_2 \quad x : \tau_1, \Gamma \vdash e_1 : \tau \quad y : \tau_2, \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\text{case } e \text{ of InL } x \rightarrow e_1; \text{InR } y \rightarrow e_2) : \tau}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2}
 \qquad
 \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst } e : \tau_1}
 \qquad
 \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd } e : \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 \ e_2 : \tau_2}
 \qquad
 \frac{x : \tau_1, \Gamma \vdash e : \tau_2}{\Gamma \vdash \lambda x. e : \tau_1 \rightarrow \tau_2}$$

Squinting a Little

Lets remove all the **terms**, leaving just the types and the contexts:

$$\begin{array}{c}
 \frac{\Gamma \vdash 0}{\Gamma \vdash \tau} \qquad \frac{}{\Gamma \vdash 1} \\
 \\
 \frac{\Gamma \vdash \tau_1}{\Gamma \vdash \tau_1 + \tau_2} \qquad \frac{\Gamma \vdash \tau_2}{\Gamma \vdash \tau_1 + \tau_2} \\
 \\
 \frac{\Gamma \vdash \tau_1 + \tau_2 \quad \tau_1, \Gamma \vdash \tau \quad \tau_2, \Gamma \vdash \tau}{\Gamma \vdash \tau} \\
 \\
 \frac{\Gamma \vdash \tau_1 \quad \Gamma \vdash \tau_2}{\Gamma \vdash \tau_1 \times \tau_2} \qquad \frac{\Gamma \vdash \tau_1 \times \tau_2}{\Gamma \vdash \tau_1} \qquad \frac{\Gamma \vdash \tau_1 \times \tau_2}{\Gamma \vdash \tau_2} \\
 \\
 \frac{\Gamma \vdash \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash \tau_1}{\Gamma \vdash \tau_2} \qquad \frac{\tau_1, \Gamma \vdash \tau_2}{\Gamma \vdash \tau_1 \rightarrow \tau_2}
 \end{array}$$

Does this resemble anything you've seen before?

A surprising coincidence!

Types are exactly the same structure as *constructive logic*:

$$\begin{array}{c}
 \frac{\Gamma \vdash \perp}{\Gamma \vdash P} \quad \frac{}{\Gamma \vdash \top} \\
 \\
 \frac{\Gamma \vdash P_1}{\Gamma \vdash P_1 \vee P_2} \quad \frac{\Gamma \vdash P_2}{\Gamma \vdash P_1 \vee P_2} \\
 \\
 \frac{\Gamma \vdash P_1 \vee P_2 \quad P_1, \Gamma \vdash P \quad P_2, \Gamma \vdash P}{\Gamma \vdash P} \\
 \\
 \frac{\Gamma \vdash P_1 \quad \Gamma \vdash P_2}{\Gamma \vdash P_1 \wedge P_2} \quad \frac{\Gamma \vdash P_1 \wedge P_2}{\Gamma \vdash P_1} \quad \frac{\Gamma \vdash P_1 \wedge P_2}{\Gamma \vdash P_2} \\
 \\
 \frac{\Gamma \vdash P_1 \rightarrow P_2 \quad \Gamma \vdash P_1}{\Gamma \vdash P_2} \quad \frac{P_1, \Gamma \vdash P_2}{\Gamma \vdash P_1 \rightarrow P_2}
 \end{array}$$

This means, if we can construct a **program** of a certain **type**, we have also created a constructive **proof** of a certain **proposition**.

The Curry-Howard Isomorphism

This correspondence goes by many names, but is usually attributed to **Haskell Curry** and **William Howard**.

It is a *very deep* result:

Programming	Logic
Types	Propositions
Programs	Proofs
Evaluation	Proof Simplification

The Curry-Howard Isomorphism

This correspondence goes by many names, but is usually attributed to **Haskell Curry** and **William Howard**.

It is a *very deep* result:

Programming	Logic
Types	Propositions
Programs	Proofs
Evaluation	Proof Simplification

It turns out, no matter what logic you want to define, there is always a corresponding λ -calculus, and vice versa.

Constructive Logic	Typed λ -Calculus
Classical Logic	Continuations
Modal Logic	Monads
Linear Logic	Linear Types, Session Types
Separation Logic	Region Types

Examples

Example (Commutativity of Conjunction)

$andComm :: A \times B \rightarrow B \times A$

$andComm\ p = (snd\ p, fst\ p)$

This proves $A \wedge B \rightarrow B \wedge A$.

Examples

Example (Commutativity of Conjunction)

$$\begin{aligned} \text{andComm} &:: A \times B \rightarrow B \times A \\ \text{andComm } p &= (\text{snd } p, \text{fst } p) \end{aligned}$$

This proves $A \wedge B \rightarrow B \wedge A$.

Example (Transitivity of Implication)

$$\begin{aligned} \text{transitive} &:: (A \rightarrow B) \rightarrow (B \rightarrow C) \rightarrow (A \rightarrow C) \\ \text{transitive } f \ g \ x &= g \ (f \ x) \end{aligned}$$

Transitivity of implication is just **function composition**.

Caveats

All functions we define have to be **total and terminating**.
Otherwise we get an **inconsistent** logic that lets us prove false things:

$$\begin{aligned} proof_1 &:: P = NP \\ proof_1 &= proof_1 \end{aligned}$$

$$\begin{aligned} proof_2 &:: P \neq NP \\ proof_2 &= proof_2 \end{aligned}$$

Most common calculi correspond to **constructive** logic, not **classical** ones, so principles like the **law of excluded middle** or **double negation elimination** do **not** hold:

$$\neg\neg P \rightarrow P$$

Inductive Structures

What about types like lists?

```
data IntList = Nil | Cons Int IntList
```

We **can't** express these in MinHS yet:

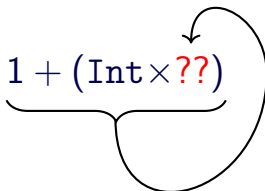
$$1 + (\text{Int} \times ??)$$

Inductive Structures

What about types like lists?

```
data IntList = Nil | Cons Int IntList
```

We **can't** express these in MinHS yet:



We need a way to do recursion!

Recursive Types

We introduce a new form of type, written $\text{rec } t. \tau$, that allows us to refer to the entire type:

$$\text{IntList} \simeq \text{rec } t. 1 + (\text{Int} \times t)$$

Recursive Types

We introduce a new form of type, written $\text{rec } t. \tau$, that allows us to refer to the entire type:

$$\begin{aligned}\text{IntList} &\simeq \text{rec } t. 1 + (\text{Int} \times t) \\ &\simeq 1 + (\text{Int} \times (\text{rec } t. 1 + (\text{Int} \times t)))\end{aligned}$$

Recursive Types

We introduce a new form of type, written `rec t.  $\tau$` , that allows us to refer to the entire type:

$$\begin{aligned}\text{IntList} &\simeq \text{rec } t. 1 + (\text{Int} \times t) \\ &\simeq 1 + (\text{Int} \times (\text{rec } t. 1 + (\text{Int} \times t))) \\ &\simeq 1 + (\text{Int} \times (1 + (\text{Int} \times (\text{rec } t. 1 + (\text{Int} \times t))))) \\ &\simeq \dots\end{aligned}$$

Typing Rules

We construct a recursive type with `roll`, and unpack the recursion one level with `unroll`:

$$\frac{??}{\Gamma \vdash \text{roll } e : \text{rec } t. \tau}$$

Typing Rules

We construct a recursive type with `roll`, and unpack the recursion one level with `unroll`:

$$\frac{\Gamma \vdash e : \tau[t := \text{rec } t. \tau]}{\Gamma \vdash \text{roll } e : \text{rec } t. \tau}$$

Typing Rules

We construct a recursive type with `roll`, and unpack the recursion one level with `unroll`:

$$\frac{\Gamma \vdash e : \tau[t := \text{rec } t. \tau]}{\Gamma \vdash \text{roll } e : \text{rec } t. \tau}$$

$$\frac{\Gamma \vdash e : \text{rec } t. \tau}{\Gamma \vdash \text{unroll } e : \tau[t := \text{rec } t. \tau]}$$

Example

Example

Take our IntList example:

$$\text{rec } t. 1 + (\text{Int} \times t)$$

`[]` =

Example

Example

Take our IntList example:

$$\text{rec } t. 1 + (\text{Int} \times t)$$
$$\begin{aligned} [] &= \text{roll } (\text{InL } ()) \\ [1] &= \end{aligned}$$

Example

Example

Take our IntList example:

$$\text{rec } t. 1 + (\text{Int} \times t)$$
$$\begin{aligned} [] &= \text{roll } (\text{InL } ()) \\ [1] &= \text{roll } (\text{InR } (1, \text{roll } (\text{InL } ()))) \\ [1, 2] &= \end{aligned}$$

Example

Example

Take our IntList example:

$$\text{rec } t. 1 + (\text{Int} \times t)$$
$$[] = \text{roll } (\text{InL } ())$$
$$[1] = \text{roll } (\text{InR } (1, \text{roll } (\text{InL } ())))$$
$$[1, 2] = \text{roll } (\text{InR } (1, \text{roll } (\text{InR } (2, \text{roll } (\text{InL } ())))))$$

Dynamic Semantics

Nothing interesting here:

$$\frac{e \mapsto_M e'}{\text{roll } e \mapsto_M \text{roll } e'} \quad \frac{e \mapsto_M e'}{\text{unroll } e \mapsto_M \text{unroll } e'}$$

$$\frac{}{\text{unroll } (\text{roll } e) \mapsto_M e}$$